

Reporte Técnico: Desarrollo de API REST con Arquitectura Limpia en Spring Boot

Presentada por: Juan Antonio Silva Flores | Junio de 2026

1. Objetivo del Proyecto

El propósito de este proyecto es diseñar y desarrollar una API REST funcional que implemente operaciones GET y POST. La solución actúa como un puente de integración, consumiendo a su vez una API de terceros para centralizar la gestión de datos. Este entregable sirve para evaluar los parámetros de calidad en el desarrollo de software, tales como el diseño arquitectónico, la elaboración de documentación formal y la cobertura de pruebas automatizadas.

2. Configuración del Entorno y Arquitectura

A continuación, se detallan las decisiones técnicas tomadas para la inicialización y estructuración del proyecto.

2.1. Generación del Proyecto y Dependencias

Para la inicialización del proyecto se utilizó Spring Initializr, configurando un entorno robusto basado en las siguientes tecnologías principales:

Proyecto: Gradle - Groovy

Lenguaje: Java 17

Framework: Spring Boot 4.0.6

Dependencias Core:

- **Spring Web:** Para la creación de servicios RESTful y el uso del contenedor embebido Apache Tomcat.
- **H2 Database:** Base de datos en memoria para persistencia rápida y ligera en ambientes de desarrollo.
- **SpringDoc OpenAPI:** Para la generación automática de la documentación de los endpoints (Swagger UI).
- **Spring Boot DevTools:** Herramientas de productividad para el desarrollador (LiveReload).

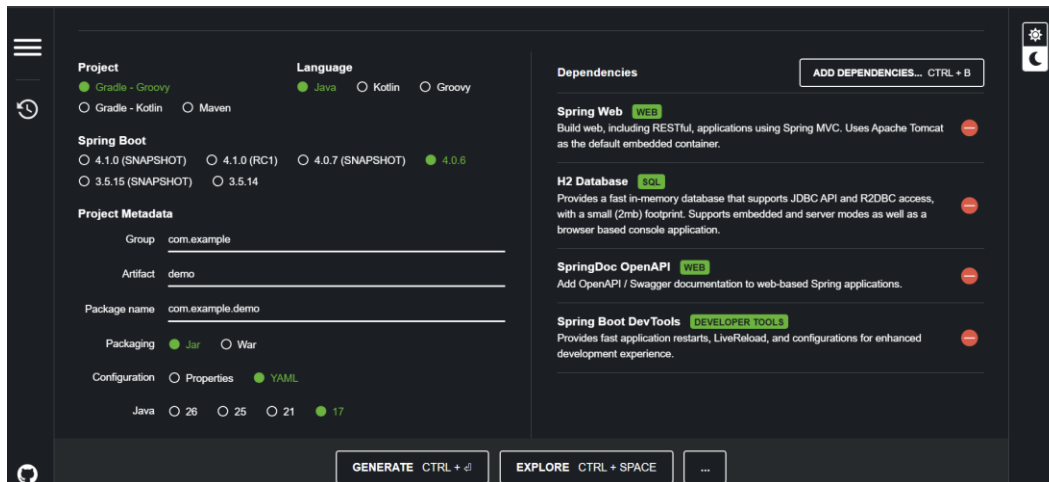


Imagen 1: Dependencias para el proyecto, configuración de build y versión de Java

2.2. Diseño de Arquitectura por Capas

Se implementó un patrón de diseño por capas (Clean Architecture) con el fin de garantizar una separación clara de responsabilidades, facilitar la transferencia de conocimiento, asegurar la mantenibilidad a largo plazo y permitir el desacoplamiento del código.

- **client:** Módulos de comunicación externa y consumo de servicios.
- **config:** Configuraciones globales de la aplicación y definición de beans.
- **controller:** Puntos de entrada HTTP (Exposición de la API REST).
- **model:** Entidades de negocio y objetos de transferencia de datos (DTOs).
- **repository:** Capa de persistencia y acceso a datos local.
- **service:** Orquestación y lógica de negocio core.

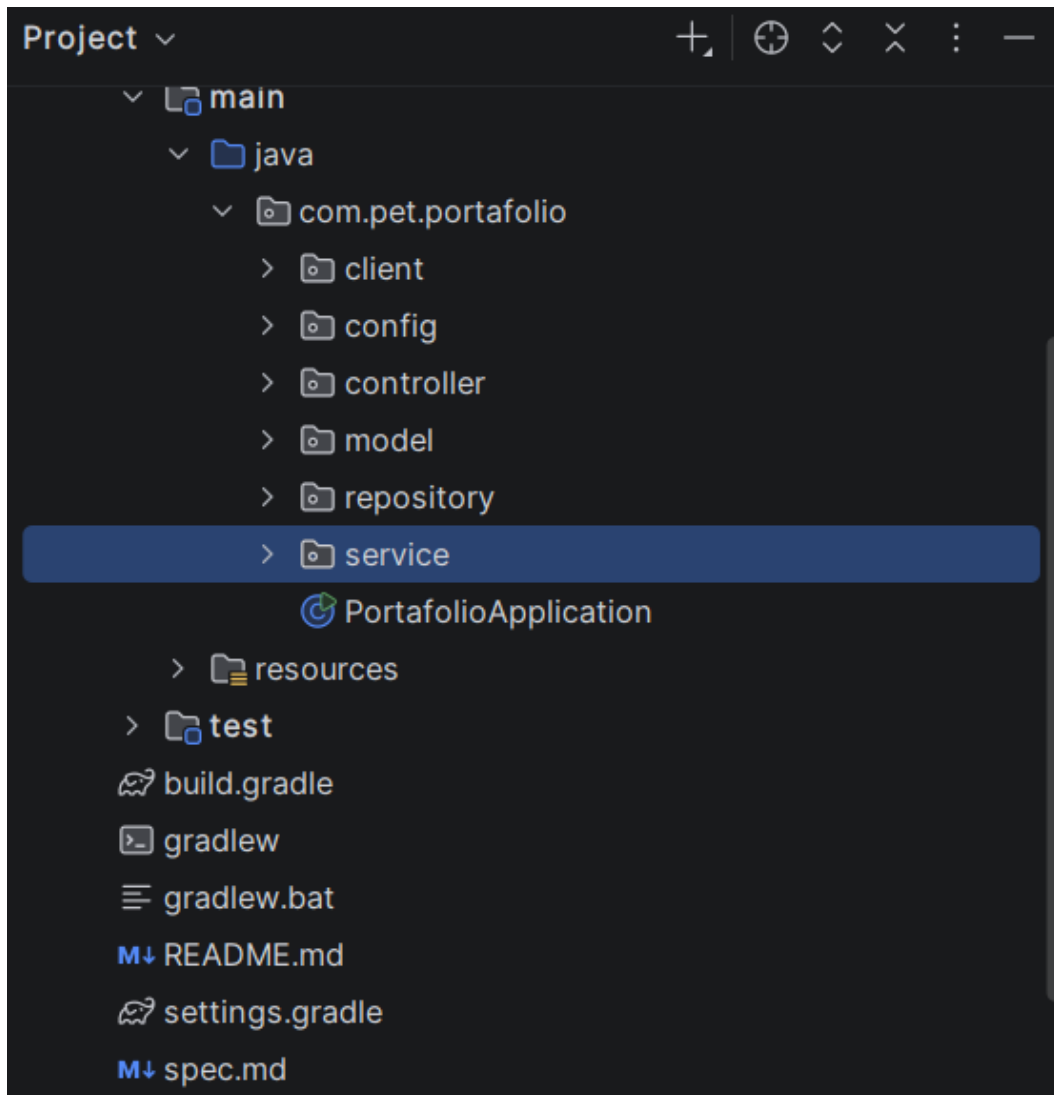


Imagen 2: Estructura por capas para una arquitectura limpia

3. Implementación del Código (Estructura de Capas)

A continuación se muestra de manera secuencial la implementación del código fuente correspondiente a cada componente de la arquitectura:

Capa de Cliente (client & config)

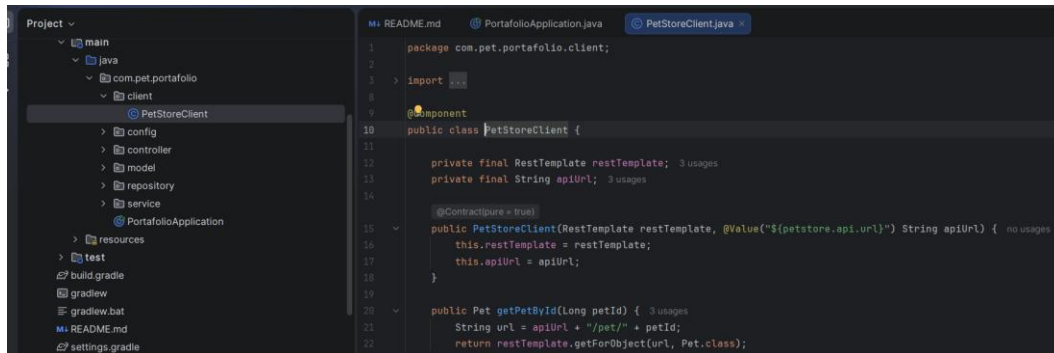


Imagen 3: Capa Client - Implementación de RestTemplate

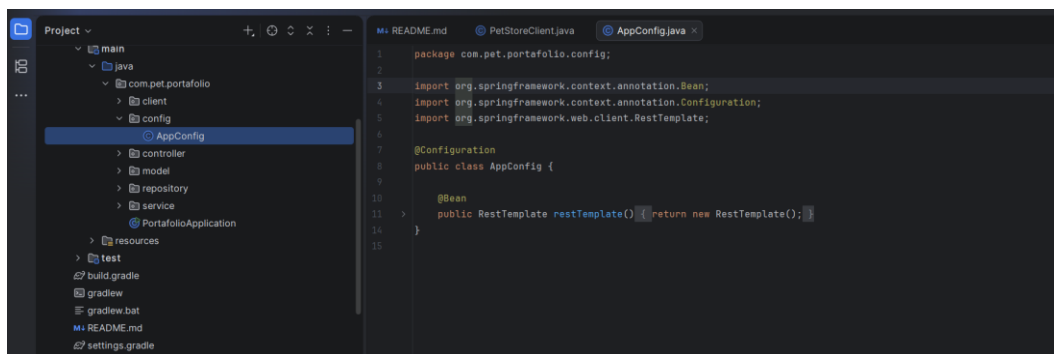


Imagen 4: Capa Config - Definición de Beans en AppConfig

Capa de Controladores (controller)

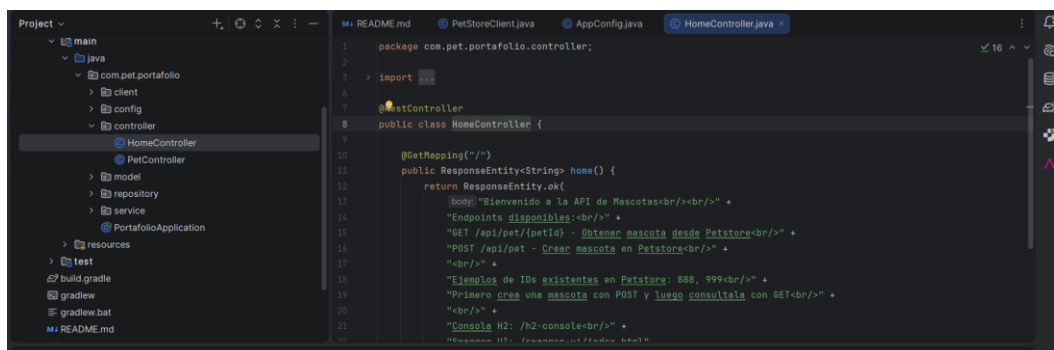


Imagen 5: Capa Controller - Endpoints HTTP expuestos

Capa de Modelos (model)

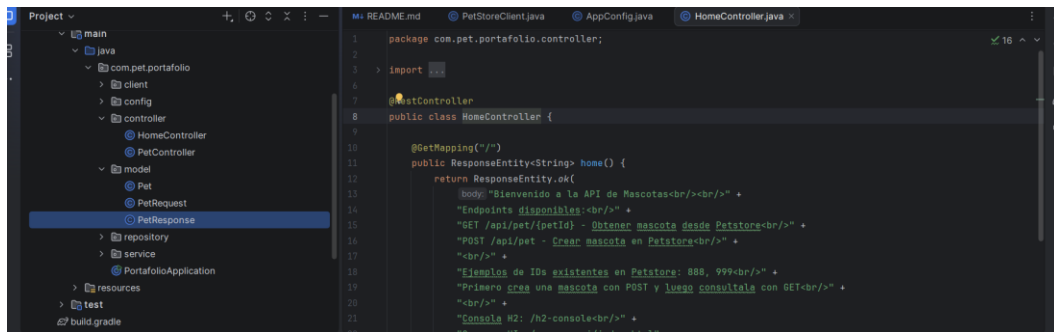


Imagen 6: Capa Model - Estructura de Entidades y DTOs

Capa de Persistencia (repository)

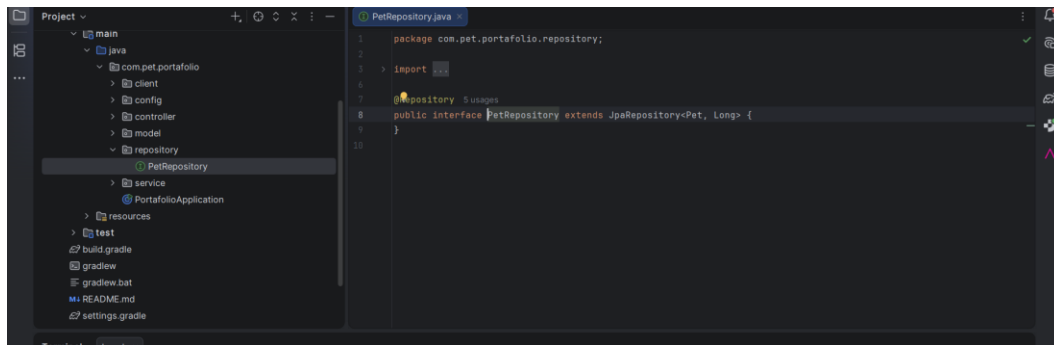


Imagen 7: Capa Repository - Interfaz JpaRepository

Capa de Negocio (service)

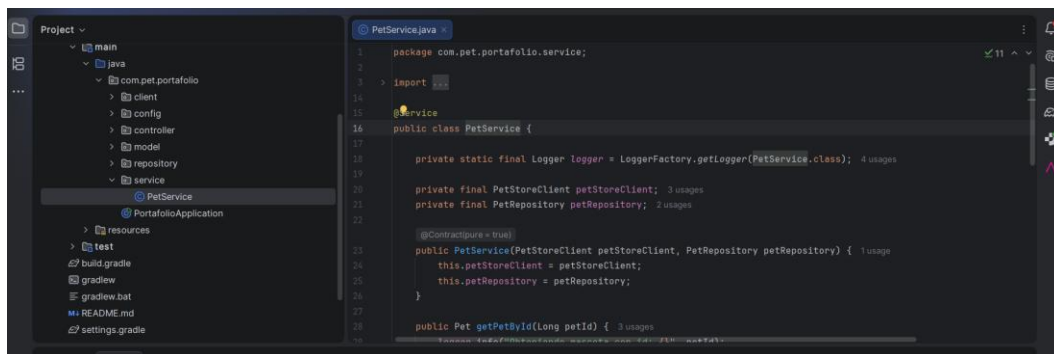


Imagen 8: Capa Service - Lógica de negocio core de PetService

4. Estrategia de Pruebas (Testing)

El proyecto cuenta con un enfoque de pruebas enfocado en la validación del comportamiento de cada capa de forma aislada.

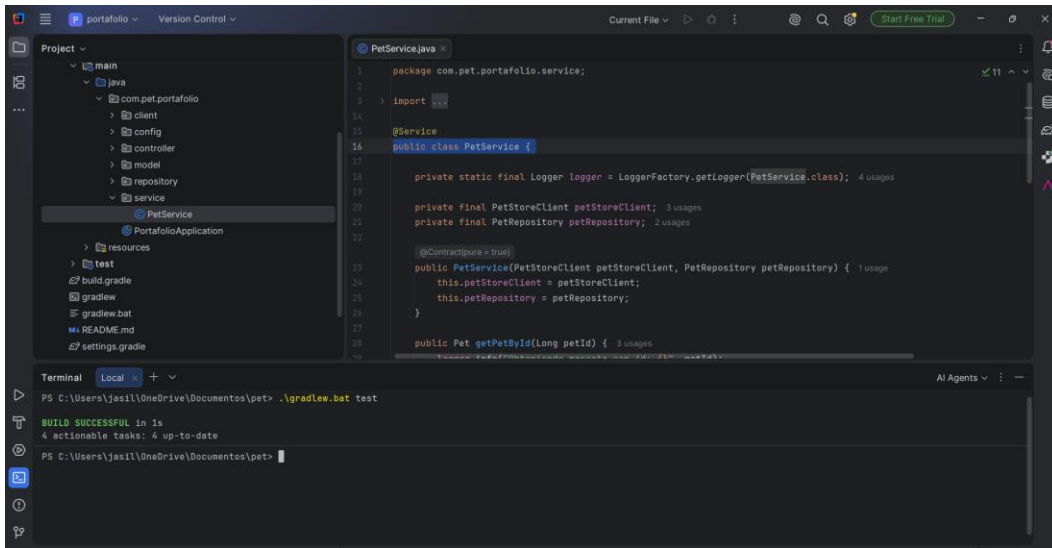


Imagen 9: Ejecución exitosa de pruebas automatizadas mediante Gradle (BUILD SUCCESSFUL)

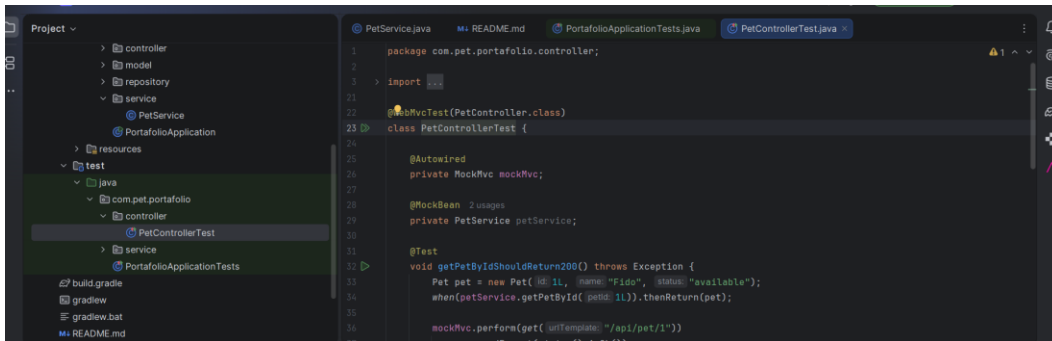


Imagen 10: Pruebas unitarias aisladas de la Capa Controller utilizando MockMvc y @MockBean

5. Verificación y Evidencias de Funcionamiento

5.1. Prueba del Método POST vía Postman

Prueba de carga útil (Payload JSON) en Postman para la creación exitosa de una entidad con respuesta 201 Created:

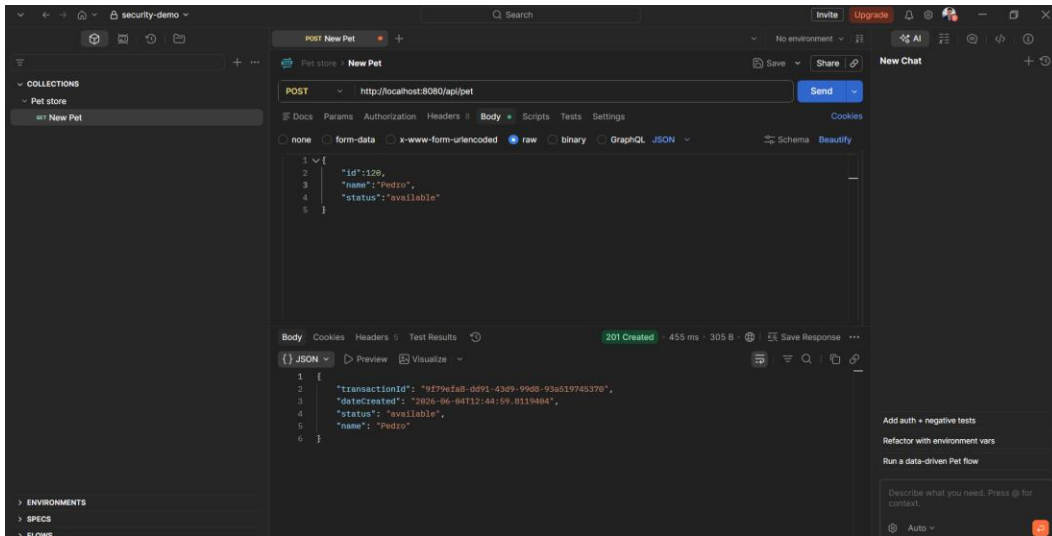


Imagen 11: Envío de POST en Postman y retorno de campos calculados (transactionId, dateCreated)

5.2. Consumo de API de Terceros y Endpoint GET

Validación del consumo síncrono del servicio externo e integración de la respuesta:

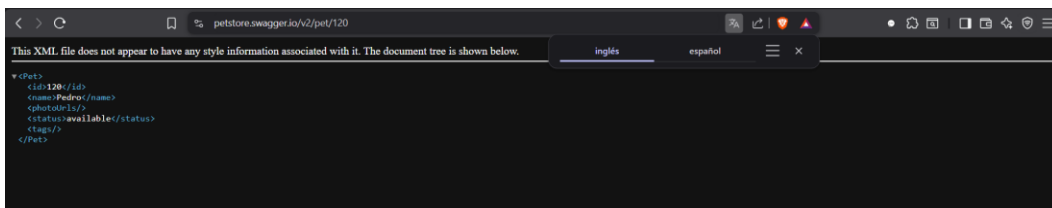


Imagen 12: Petición GET consumiendo datos unificados desde la API de Swagger de terceros

5.3. Documentación Viva con Swagger UI

Generación automática e interactiva de la documentación de la API mediante OpenAPI:

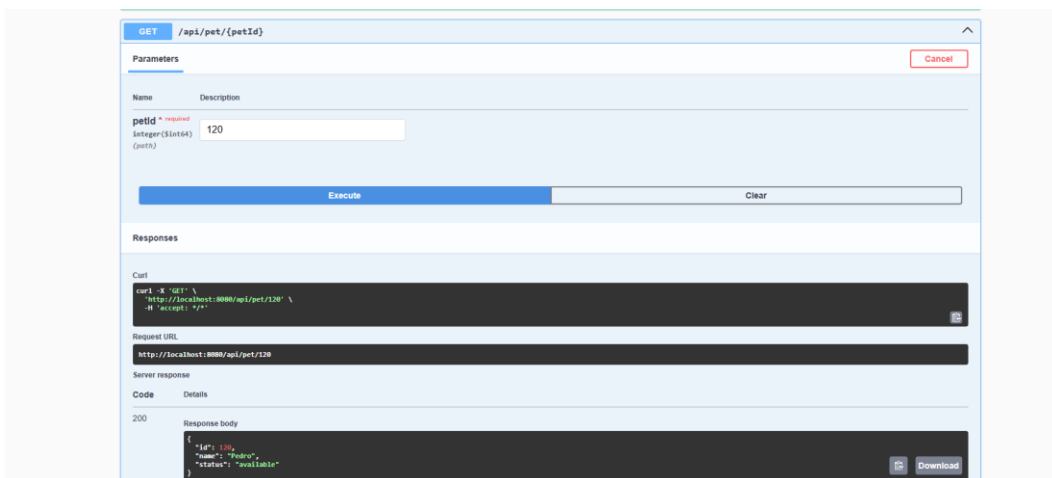


Imagen 13: Interfaz gráfica interactiva de Swagger UI para pruebas dinámicas de endpoints

6. Conclusiones de Entrega

La API cumple de manera estricta con todos los requerimientos propuestos en el objetivo inicial: arquitectura limpia por capas, integración correcta con servicios de terceros, cobertura de pruebas unitarias automatizadas aisladas y documentación autogenerada interactiva. Queda lista para su posterior despliegue y evaluación final.